Talent Trail

Final Team Presentation

Sarah Suliman:

A Career Gap Analysis Tool

Course:
Software Development Tools & Methods

- Mark Olmscheid
- Kassidy Flick
- Lisa Wilder
- Sarah Suliman

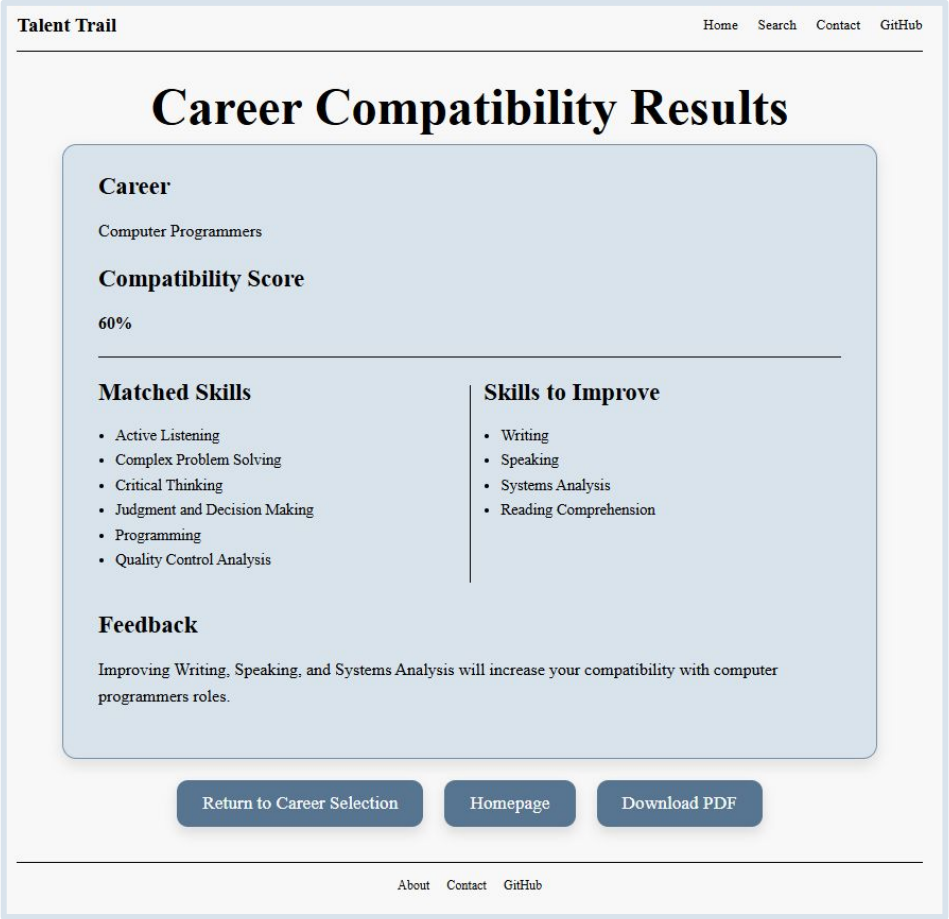
Problem Motivation

- Many people struggle to accurately evaluate their skills and readiness for a desired career
  - Determining the steps needed to reach a dream job can feel overwhelming
  - Online information is often inconsistent and difficult to navigate

Goal:

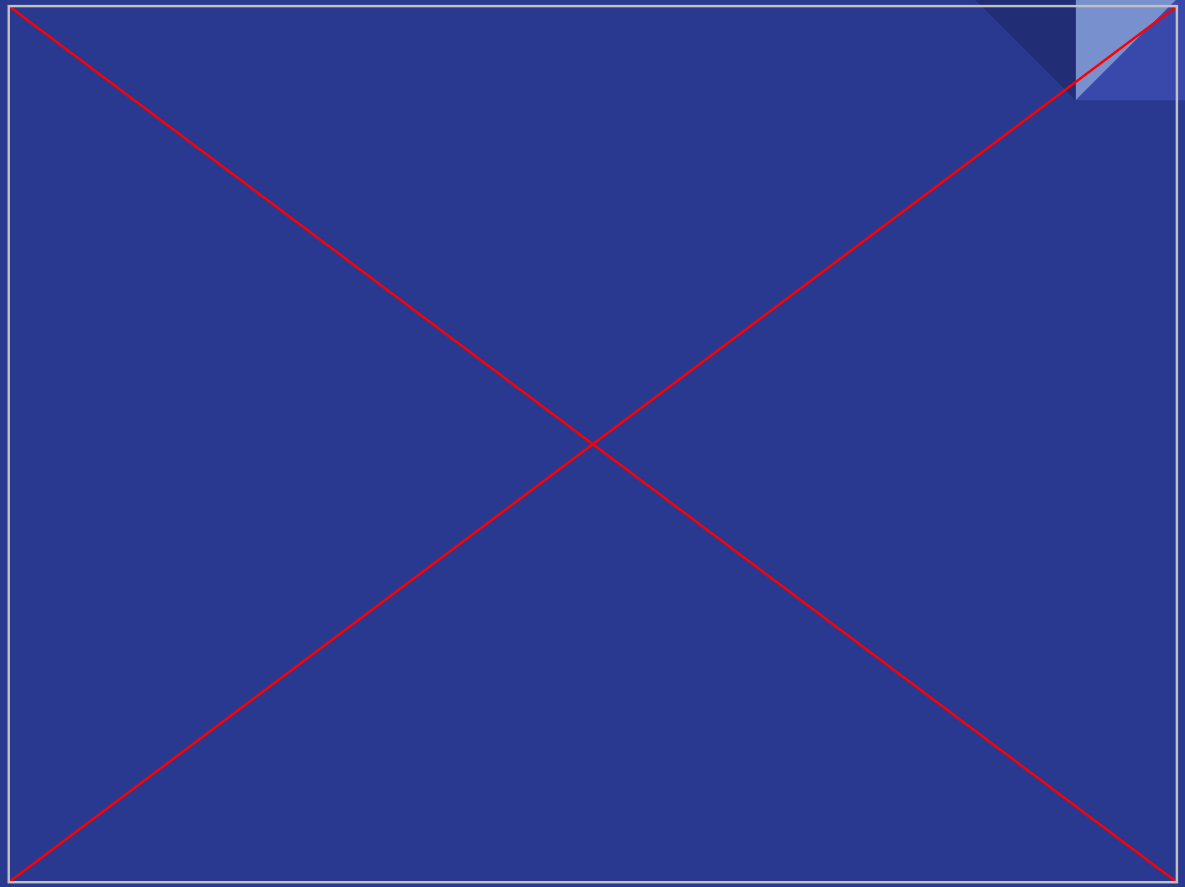
Leverage real-world data from O*NET to compare a user's current skills with career requirements and generate a clear report identifying skill gaps and actionable steps toward potential careers.

- Talent Trail enables users to:
  - Select a desired career
  - Self-assess their skills relevant to that career
  - Receive a match percentage
  - Compare their current skills to O*NET skill expectations
  - Generate a report highlighting skill gaps and areas for improvement



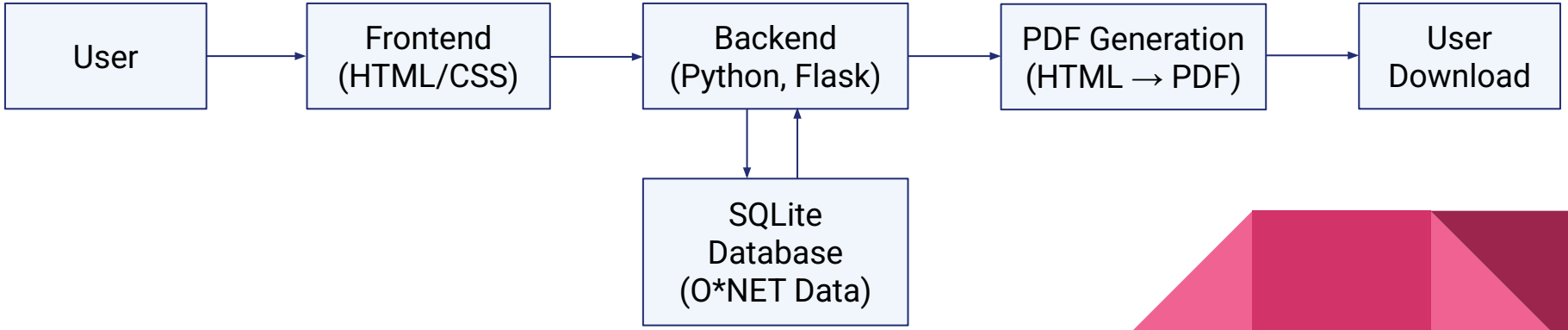
Demo Flow Overview

1. User lands on the homepage
2. User navigates to career selection
3. User selects a career from a dropdown menu
4. The system retrieves:
   - Career title
   - Top importance-ranked skills for the selected career
5. User completes a skills survey (rated on a 0–7 scale)
5. The system:
   - Calculates a compatibility score
   - Identifies skill gaps
6. Results page displays:
   - Compatibility score (percentage)
   - Matched skills
   - Skills to Improve
7. User clicks a button to generate a PDF report
8. PDF report is generated and downloaded



System Architecture Overview

- Frontend: HTML, CSS
- Backend: Python, Flask
- Database: SQLite, O*NET



Tools Used

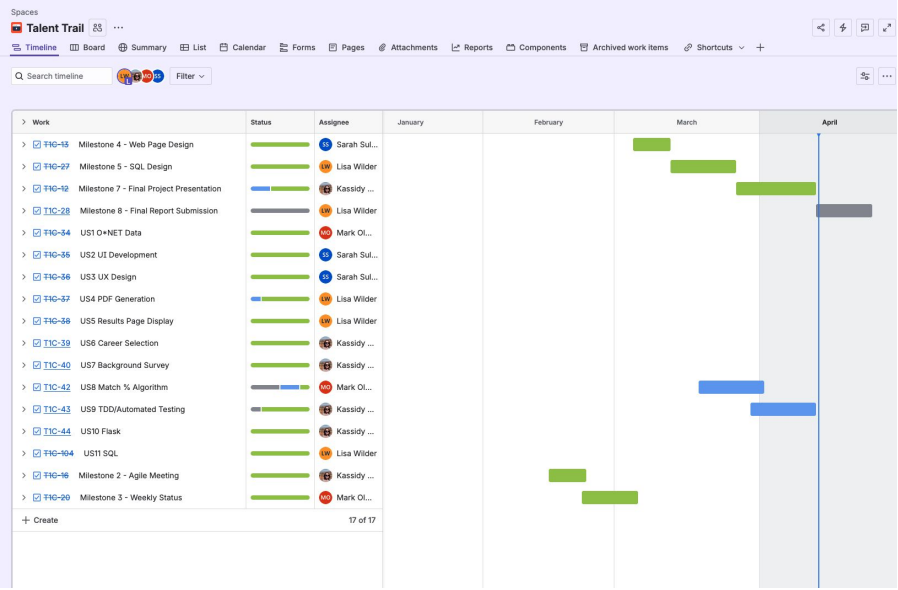
Tool	Purpose	Evaluation
Python	Backend logic, API, skill gap calculations	**Excellent:** Flexible, well-supported
Flask	Web server for routing and templating	**Excellent:** Lightweight and easy to integrate
SQLite	Store and query O*NET data	**Good:** Simple for MVP, but large dataset posed performance challenges
HTML/CSS	Frontend layout and styling	**Excellent:** Easy to customize
WeasyPrint	PDF report generation	**Good:** Worked well once configured, learning curve was steep
GitHub	Version control, collaboration	**Excellent:** Crucial for managing code and merge conflicts
JupyterHub	Development and testing	**Mixed:** Proxy URL and dataset size caused issues

Lisa Wilder:

Tools Used

★ Jira Board:

- Track tasks, ownership and team progress
- Communication about tasks
- Visual representation of status of project
- Initially the board had a minimal framework that grew as the project evolved



Kassidy Flick:

Key Features Implemented

- ★ Career Selection & Skill Retrieval
  - Browse Careers from the O*NET Database
  - Automatically fetch the most important skills for selected career
- ★ Personalized Skill Survey
  - Users rate their proficiency in each skill (0-7)
- ★ Feedback and Guidance
  - Suggests which skills to focus on for career compatibility
- ★ Skill Gap Analysis & Compatibility Score
  - Compares user ratings to O*NET skill levels
  - Calculates a compatibility score based on if the user meets or exceeds the O*NET baseline
  - Highlights matched skills and skills that need improvement
- ★ PDF Report Generation
  - Generates a downloadable, formatted PDF of survey results and O*NET proficiency comparison

Compatibility Score

1. Users rate their proficiency in each skill (0–7 scale, matching O*NET Leveling system)
2. For each skill, the app compares the user rating to the O*NET baseline
3. If the user meets or exceeds the O*NET Level, that skill is counted as a match.

$$Compatability = \frac{Matched\ Skills}{Total\ Skills\ Rated} \times 100$$

Weighted Gap Methodology

Scale	What It Measures	Range
Importance	How crucial a skill, task, or knowledge is to the job	1–5
Level	How proficient or advanced a skill needs to be	0–7

1. Users rate their proficiency in each selected skill
2. Raw gap is calculated

$$Gap = ONET\ Level - User\ Level$$

3. Negative gaps are reset to zero (not treated as a deficiency if above O*NET Level)
4. Gap is weighted by importance

$$Weighted\ Gap = Gap \times \frac{ONET\ Importance}{5}$$

Weighted gap is used to rank skills from the highest priority deficiency to lowest.

Major Technical Challenges

- Handling Large O*NET Data
  - Processing and querying the full database efficiently on JupyterHub
- JupyterHub Proxy & URL Routing
  - Adapting Flask routes to work behind the JupyterHub service prefix
- PDF Generation
  - Learning and implementing libraries (WeasyPrint) to dynamically create reports

Kassidy Flick:

What Went Well

- Backend Functionality
  - Successfully integrated Flask with SQLite and O*NET data
  - Implemented skill gap calculation and weighted gap ranking
- User Flow & Frontend
  - Smooth career selection and survey experience
  - Clear display of match scores and skill gaps
  - Standard CSS styling throughout entire website
- Early usability improved overall design
- PDF Report Generation
  - Downloadable reports created from survey results
- Problem-Solving & Learning
  - Overcame challenges with JupyterHub proxy URLs
  - Learned new tools like WeasyPrint
- Data Handling
  - Efficient queries for top skills and career-level information

Sarah Suliman:

What Could Have Gone Better

- Time Management
  - Busy schedules caused uneven progress
- Team Communication
  - In the beginning, coordination issues led to delays and caused some miscommunications
- MVP Scope Adjustment
  - Removed previous experience and education from compatibility scoring and user reports to meet deadlines
- Version Control & Merging Conflicts
- PDF Generation
  - Implementing final report generation and design towards the end of the project, which caused some last minute minor bugs
- Large & Complex Database
  - Many tables and schemas made initial querying and integration tricky

Lessons Learned

- Time & Scope Management
  - Break work into smaller milestones to avoid last-minute pressure
  - Prioritize core MVP features over “nice-to-haves”
- Team Communication
  - Frequent check-ins and clear task assignments reduce misunderstandings
  - Version control best practices prevent merge conflicts
- Technical Insights
  - Large datasets require careful querying and memory management
  - Learning new libraries takes additional time
  - Backend design should anticipate proxy environments (like JupyterHub)

Potential Future Expansions

Feature	Description	Benefit
Incorporate Experience and Education	*Include previous work experience and education in compatibility scoring*	Produces a more holistic and accurate assessment of career readiness
Education Distribution Visualization	*Display typical education distribution for selected careers*	Helps users understand common career paths and educational requirements
Multi-Career Comparison	*Compare user skills across multiple careers*	Helps users explore alternative career paths and make informed decisions
User Accounts & Saved Reports	*Enable login and report saving for multiple sessions*	Supports long-term tracking and personalized planning
Trie Tree Career Selection	*Replace career selection dropdown with Trie Tree algorithm*	Faster and more intuitive career search. Improves UX

Sarah Suliman:

Final Reflection

- We learned how to:
  - Plan and deliver iterative milestones, adjusting the MVP as needed
  - Collaborate effectively using GitHub, including managing merge conflicts
  - Handle technical challenges with large datasets and proxy environments (JupyterHub)
  - Integrate multiple backend and frontend components in a full-stack project
  - Generate and format professional PDF reports from dynamic data
  - See a project through from idea to demo

Thank You

- Any Questions?
- GitHub:
  - https://github.com/kflick3r/Talent_Trail